

Accessing Octave's functions

In our next extension, we will look at how we can call Octave functions from our extension. To keep things simple, we will pass an Octave function name and a parameter list to our Oct-file's function, and then our extension will call the corresponding function. Save the following code in a file named *global_oct.cc*.

```
/* global_oct.cc */

#include <octave/oct.h>
#include <octave/parse.h>

DEFUN_DLD (global_oct, args, nargout, "Demo of calling Octave functions")
{
    int nargin = args.length ();
    octave_value_list retval;

    if (nargin < 2)
        print_usage ();
    else
    {
        octave_value_list newargs;

        for (octave_idx_type i = nargin - 1; i > 0; i--)
            newargs (i-1) = args(i);

        if (args(0).is_function_handle ())
        {
            octave_function *fcn = args(0).function_value();
            if (! error_state)
                retval = feval (fcn, newargs, nargout);
        }
        else
            error ("funcdemo: expected a function handle");
    }

    return retval;
}
```

Compile the file to get the Oct-file, and then play around with it:

```
octave:1> mkoctfile global_oct.cc
octave:2> global_oct(@sin,1)
ans = 0.84147
octave:3> global_oct(@sin,pi/2)
ans = 1
octave:4> mkoctfile global_oct.cc
octave:5> global_oct(@sin,pi/2)
warning: global_oct
ans = 1
octave:6> global_oct(@cos,pi/2)
ans = 6.1230e-17
octave:8> global_oct(@max,[1,2,-3])
ans = 2
```

In this code, we have asked the user to pass the function handle (denoted by @) to the program. You can also pass in-line functions, anonymous function handles, and strings. Please refer to the Octave documentation for details.

Like Octave's functions, we can also make use of Octave's data structures, like matrices and arrays, in our extensions. We discuss these next, while dealing with coding standalone C++ programs that may need to make use of Octave's data structures and/or call Octave's functions.

Creating Octave-powered standalone programs

The following code *standalone.cc* shipped with the Octave source distribution in *examples/* is a very simple example of how you can access Octave's data structures from C++ programs.

```
/* standalone.cc */

#include <iostream>
#include <octave/oct.h>

int main (void)
{
    std::cout << "Hello Octave world!\n";
    int n = 2;
    Matrix a_matrix = Matrix (n, n);
    for (octave_idx_type i = 0; i < n; i++)
    {
        for (octave_idx_type j = 0; j < n; j++)
        {
            a_matrix (i, j) = (i + 1) * 10 + (j + 1);
        }
    }
    std::cout << a_matrix;
    return 0;
}
```

Now, we want to compile and link this code with the Octave libraries (*liboctave*). *mkoctfile -link-stand-alone standalone.cc -o standalone* will compile and link the file to create a standalone executable. Execute it:

```
./standalone

Hello Octave world!
11 12
21 22
```

You may also now call any operation allowed on the matrix data type. For example, you can calculate the inverse of the matrix using this code:

```
/*standalone1.cc*/

#include <iostream>
#include <octave/oct.h>

int main (void)
```